JUNIOR PENTESTER FIELD NOTES

Vulnerability Scanning • Verification • Automation • Load Balancers

Prepared as a practical training reference — theory, mindset, and hands-on commands for authorized engagements only.

SCOPE & AUTHORIZATION FIRST

Every command, scan and workflow in this document assumes a signed authorization / rules-of-engagement covering the exact IP ranges and time windows tested. Nothing here is a substitute for written permission.

How to use this document: each module pairs the underlying theory with a practical lab drill. Read the concept box first, then run the command block against your own lab (e.g. a Metasploitable / DVWA / TryHackMe box) before moving on.

MODULE 4

Vulnerability Scanning Fundamentals

From host discovery to your first real findings

1. What a vulnerability scanner is actually doing

A vulnerability scanner is not magic — it is a large, maintained library of **checks** for known weaknesses (CVEs, misconfigurations, outdated software, default credentials) that it tests against a target and compares to observed behaviour.

Target → Scanner sends checks → Compare behaviour/config → Match known weakness? → Finding

Junior-level mental model

Think of the scanner as a huge checklist walking up to a door and trying every known key. It tells you which key *seemed* to fit — it does not tell you the door is definitely open.

2. Nmap vs a dedicated vulnerability scanner

Tool	Primary job	Typical output
Nmap	Host/port/service/version discovery	Open ports, service banners, OS guess
Nessus / OpenVAS / Nexpose	Match services against known-vulnerability database	CVE findings with severity + remediation
Nuclei / Nikto	Template-based / web-focused checks	Specific misconfig or CVE hits

LAB — Step 1: Host discovery

```
nmap -sn 192.168.56.0/24
# -sn = ping sweep, no port scan. Just "who is alive?"
```

LAB — Step 2: Service & version fingerprint

```
nmap -sV -T3 192.168.56.20
# -sV = version detection, -T3 = default/normal timing (safe default)
```

3. Broad scan first, then targeted scan

Broad scan → Interesting findings → Investigate → Targeted scan

Start wide when you don't have a hypothesis yet. Once something interesting shows up (an old Apache banner, an exposed RDP port, a login page), narrow down to a scan template built for that exact protocol or CVE family (e.g. a web-app template, or a specific Log4j / ProxyShell / PrintNightmare check).

Why not just scan everything with every check?

Broad + all-checks scans are noisier, slower, and more likely to disrupt fragile hosts. Match scan intensity to what you actually need answered.

4. A finding is not a confirmed vulnerability

Scanner finding → Understand it → Verify → Determine real impact

> **Common junior mistake**
> Seeing "HIGH — possible vulnerability" and immediately reporting it as confirmed, or immediately firing an exploit at it. Both skip the verification step covered in Module 5.

5. Practical drill — Module 4

1. Spin up a lab target (Metasploitable2 or similar).
2. Run the host-discovery command above against your lab subnet.
3. Run the version-detection command against the one live host you found.
4. Write down, in your own words, what service + version Nmap reported — that's the hypothesis your vulnerability scanner will test next.

MODULE 5

Verification & Credentialed vs Non-Credentialed Scanning

Turning a scanner finding into a pentester conclusion

1. Finding ≠ confirmed vulnerability

Every finding needs an evidence check before it goes in a report. Ask, in order:

1. What exactly did the scanner detect (banner version? response header? behaviour)?
2. Why does it believe this indicates a vulnerability?
3. Is that evidence strong (exact version match, verified response) or weak (guessed from a banner)?
4. Could this be a false positive (e.g. back-ported patch, custom banner)?
5. Can I safely verify it within scope (manual check, PoC, credentialed re-scan)?

Scan → Finding → Read it → Understand it → Evidence strong? Verify → Evidence weak? Investigate more

2. False positives — why they happen

Cause	Example
Back-ported security patch	Vendor patches an old version number without bumping it
Custom / stripped banner	Admin edited the server banner to hide the real version
Network middlebox interference	A WAF or proxy alters responses the scanner reads
Scanner plugin bug / outdated signature	Signature matches a similar but unrelated service

3. Credentialed vs non-credentialed scans

	Non-credentialed	Credentialed
Access level	None — network view only	Authorized login (local/domain account)
Answers the question	"What can an outside attacker see?"	"What risk exists once someone has valid access?"
Typical findings	Open ports, banners, exposed services	Missing patches, weak local configs, installed software inventory, registry/file checks
Noise level	Lower depth, fewer false positives from guessing	Deeper, more accurate, but requires safe credential handling

Neither is "better" — they answer different questions
A client asking about external exposure needs a non-credentialed view. A client asking "what if an employee's laptop is compromised" needs a credentialed view. Pick the scan type that matches the question you were hired to answer.

4. Verification workflow (the professional loop)

SCAN → Finding detected → Read & understand → Evidence check → Verify safely → Confirmed / False positive → Document

5. Attacker-mindset checkpoint

Scenario
Scanner reports: **HIGH — possible outdated Apache** on 192.168.50.20. You have no credentials yet. What's your first move?

Correct answer: C — understand the finding and look for evidence to verify it (banner cross-check, safe version probe, changelog comparison) *before* treating it as confirmed or attempting exploitation.

6. Practical drill — Module 5

1. Take one finding from your Module 4 scan and write a 3-line "evidence log": what was detected → why it was flagged → what would confirm or kill it.
2. If you have lab credentials, re-run the same scan credentialed and compare the extra detail you now get (patch level, installed packages) vs the non-credentialed pass.

MODULE 6

Automation

Making the methodology repeatable — GUI, CLI, and API

1. Why automate

Benefit	What it means in practice
Faster	No manual re-entry of the same scan settings for every target
Repeatable	Same workflow every week/engagement — easy to hand off to a teammate
Consistent	Reduces the chance of forgetting a step (e.g. skipping a credential set)
Customizable	Same skeleton script adapted per client scope/policy

2. Three levels of automation

GUI scheduling → CLI scripting → API integration

GUI automation — most commercial scanners (Nessus, OpenVAS/GVM, Qualys) let you save a scan policy and schedule it, e.g. every Monday at 22:00.

CLI automation — wrap the scanner's command-line interface in a shell or Python script and loop it over a target list.

LAB — CLI loop over a target list

```
cat targets.txt
# 192.168.50.20
# 192.168.50.30

while read -r ip; do
echo "[*] Scanning $ip"
nmap -sV -oN "scans/${ip}.txt" "$ip"
done < targets.txt
```

API automation — most enterprise scanners expose a REST API so a Python script can launch scans and pull results programmatically.

LAB — Nessus REST API skeleton (Python)

```
import requests

NESSUS_URL = "https://127.0.0.1:8834"
headers = {"X-ApiKeys": "accessKey=...; secretKey=..."}

# 1. Create a scan from an existing policy/template
resp = requests.post(f"{NESSUS_URL}/scans", headers=headers, verify=False, json={
"uuid": "<template-uuid>",
"settings": {"name": "Weekly-Scope-A", "text_targets": "192.168.50.20,192.168.50.30"}
})
scan_id = resp.json()["scan"]["id"]

# 2. Launch it
requests.post(f"{NESSUS_URL}/scans/{scan_id}/launch", headers=headers, verify=False)

# 3. Poll status, then pull the report once status == 'completed'
```

OpenVAS/GVM equivalent

OpenVAS exposes the same idea through **gvm-cli** / the GMP XML API — same pattern: create target → create task → start task → poll → export report.

3. Automation does NOT replace judgement

Bad: Automation → Run everything → Trust blindly → Done

Good: Human plan → Automation → Results → Human analysis → Verify → Report

Rule to tattoo on your brain

Automation handles repetition. It never handles judgement, scope decisions, or verification — that's still you.

4. Practical drill — Module 6

1. Write a 5-line bash loop that runs `nmap -sV` across a small targets.txt and saves one output file per host.
2. Sketch (pseudocode is fine) how you'd extend it to call a scanner's API instead of the CLI, for the same target list.

MODULE X1

Load Balancers — Detection & Scanning Strategy

Why LBs break your assumptions, and how to scan around them

1. Why load balancers matter to a scan

A load balancer (LB) sits in front of multiple real backend servers and distributes traffic between them. If you don't account for it, you can end up scanning "one target" that is actually 3–10 different physical/virtual servers behind one IP or hostname — and your findings might apply to only one backend, not all of them.

You → Load balancer (VIP) → Backend 1 / Backend 2 / Backend 3

2. Detecting a load balancer is in front of your target

- **Multiple/rotating server headers or session cookies** — repeated requests return slightly different *Server*, *X-Powered-By*, or session cookie values.
- **DNS returns multiple A records** for the same hostname (round-robin DNS).
- **TTL/response-time jitter** between otherwise identical requests.
- **Known LB fingerprints** — headers like *Via*, *X-Forwarded-For* handling, or classic banners (F5 BIG-IP, HAProxy, AWS ELB/ALB, NGINX, Citrix NetScaler).

LAB — Quick LB / CDN fingerprint check

```
# Compare headers across repeated requests
for i in 1 2 3 4 5; do curl -s -D - -o /dev/null https://target.tld | grep -Ei
'server|via|x-'; done

# Check for multiple DNS records (possible round-robin / GSLB)
dig target.tld A +short

# Purpose-built tool
wafw00f https://target.tld # detects WAF/LB fingerprints
lbd target.tld # 'Load Balancing Detector' — dedicated tool
```

3. Strategy once you confirm a load balancer

1. **Map the backends, don't just scan the VIP.** Ask the client for the real backend IPs in scope, or identify them via DNS history, cert SANs, or leaked internal hostnames.
2. **Scan each backend individually when authorized** — findings and patch levels can differ per node, especially during rolling updates.
3. **Watch for session-based routing (sticky sessions)** — some checks (auth flows, state-dependent bugs) only reproduce consistently if you stay pinned to one backend (e.g. reuse the same session cookie / Host header per test run).
4. **Rate-limit and distribute your testing pattern.** LBs plus WAFs in front of them often trigger on scan-speed patterns — throttle to avoid blacklisting mid-engagement.

5 **Test the LB itself as an asset** — management interfaces (e.g. F5 iControl/TMUI, NetScaler admin) are historically high-value CVEs (e.g. NetScaler ADC RCE chains) and should be in scope for a dedicated check, not skipped as "just plumbing".

> **Don't report an LB finding as "the server"**
> If Backend-2 is patched and Backend-1 isn't, and you only ever hit the VIP, your report may under- or over-state real risk. Always note in your methodology whether findings were confirmed per-backend or only observed through the VIP.

4. Practical drill — Load Balancers

1 Run the header-comparison loop above against any public site you're authorized to test (or your own lab reverse proxy) and note whether headers/cookies changed between requests.

2 Install and run `lbd` or `wafw00f` against a lab target sitting behind an NGINX reverse proxy you set up yourself, and confirm it correctly flags the proxy.

MODULE X2

Additional Tools Worth Knowing

Rounding out the scanner-only toolbox

1. Vulnerability scanners beyond Nessus

Tool	Best for	Notes
OpenVAS / GVM	Free, self-hosted general vuln scanning	Good Nessus alternative to learn on a home lab
Nuclei	Fast, template-based web/CVE checks	Huge community template library; great for automation
Qualys VMDR	Enterprise-scale, cloud-managed scanning	Common in large corporate environments
Rapid7 InsightVM	Enterprise scanning + risk scoring	Strong API + integrates with Metasploit
Nikto	Web-server misconfig scanning	Lightweight, quick first pass on a web target

2. Specialized checks that complement a general scanner

Tool	Purpose
testssl.sh / sslyze	Deep TLS/SSL configuration + cipher + certificate checks
wafw00f	Identify WAF/CDN in front of a target (affects scan strategy)
lbd	Dedicated load-balancer detection (DNS + header based)
Amass / Subfinder	Subdomain & attack-surface discovery before you even scan
Gobuster / ffuf	Content/directory discovery on web targets
CrackMapExec / NetExec	Credentialed checks & lateral-movement recon on Windows/AD
Metasploit Framework	Manual verification / controlled exploitation after a finding is confirmed

How to choose

Use a general scanner (Nessus/OpenVAS/Nuclei) to get broad coverage fast, then reach for a specialized tool to go deep on whatever category the finding falls into (TLS, WAF, AD, web content).

3. Practical drill — Tooling

1. Run `testssl.sh` against a lab HTTPS endpoint and read the summary — note any weak ciphers or expired certs it flags.

2 Run `nuclei -u -t cves/` against your lab and compare the findings to what your general scanner reported for the same host.

MODULE ★

Full Methodology Cheat Sheet

The complete loop, plus one worked example

1. The complete loop

1. Authorization & scope — confirm written permission, IP ranges, time windows.
2. Host discovery — find what's alive (Nmap ping sweep).
3. Service/version enumeration — what's running, on what port, what version.
4. Classify targets — normal vs fragile (ICS, IoT, legacy) vs load-balanced.
5. Plan protocol/topology & safety — throttling, scheduling, avoiding fragile systems.
6. Run the vulnerability scan — broad first, then targeted on interesting hits.
7. Analyze findings — what, why, how confident.
8. Verify — evidence check, credentialed re-check where useful, confirm or discard.
9. Document & report — per-backend/per-host clarity, remediation guidance.
10. Automate the repeatable parts for next time.

2. Worked example — mixed environment under a tight window

Scenario
Authorized scope: Web server (.20), Database (.30), fragile ICS (.40), specialized IoT (.50). Assessment window 22:00–02:00. ICS is extremely fragile, IoT is specialized.

1. **Confirm scope & constraints in writing** — lock the 22:00–02:00 window and flag .40/.50 as "handle with care" before touching anything.
2. **Discover safely** — light ping sweep across all four; treat .40 and .50 with a slower, less intrusive discovery pass (fewer parallel probes).
3. **Classify** — .20/.30 go into the "normal, automatable" bucket; .40 (ICS) and .50 (IoT) go into the "manual, vendor-aware" bucket.
4. **Check for a load balancer / reverse proxy in front of the web server** — since .20 exposes HTTP/HTTPS, run the header/DNS checks from the LB module before assuming it's a single backend.
5. **Run standard scans on .20/.30 inside the window**, throttled to avoid saturating the link during the live hours the client allowed.
6. **Handle .40/.50 manually** — passive/low-rate checks only, ideally read-only protocol queries, coordinated with the client's own engineers if it's ICS.
7. **Analyze every finding before trusting it** — especially on the web server, note whether a finding was confirmed against the VIP only or against an identified backend.
8. **Verify high-severity items** with evidence before reporting; keep ICS/IoT findings purely observational unless the client explicitly authorizes deeper testing there.
9. **Report with per-asset clarity** — call out which findings are confirmed vs observed-only, and which asset class (normal / fragile / load-balanced) each came from.

The one-line version of everything in this document

Scan broadly, verify before you believe it, treat credentials and load balancers as changing what "the target" even means, and automate only the parts that don't need your judgement.

Personal training reference — build on it with your own lab notes as you progress through OSCP/PNPT-style material.